

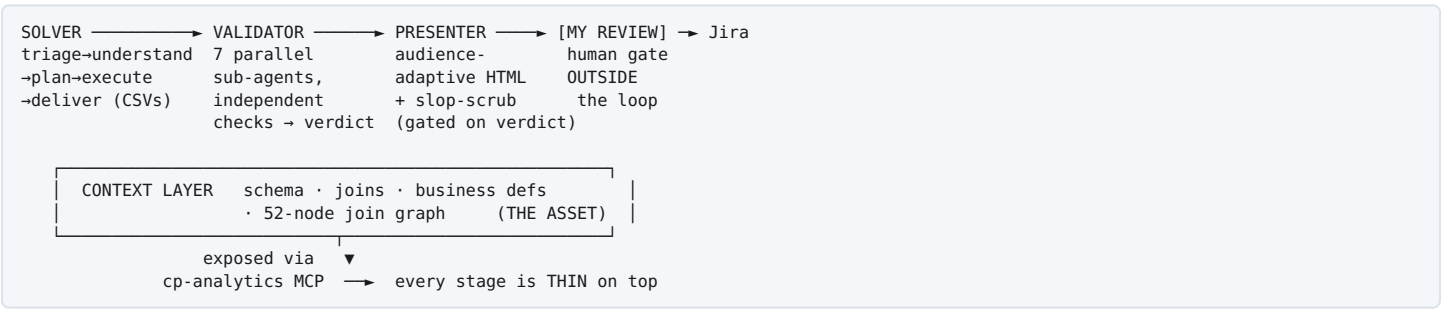
Autonomous Jira Ticket Pipeline

LEAD BUILD #1

A **3-stage autonomous pipeline** — solve → validate → present — that takes a BDV Jira ticket from raw text to a validated, stakeholder-ready deliverable with **zero internal checkpoints**.

400+ tickets since Jan '25 · 30–60 min → ~10 min · ~17–60 workdays saved

THE ARCHITECTURE — 3 SKILLS ON ONE REUSABLE SUBSTRATE



FOLDER CONTRACT — READ-ONLY INPUTS FLOW DOWNSTREAM, EACH STAGE APPENDS
tickets/BDV-XXXX/ ticket.json · ticket.md · attachments/ [input, read-only]
Solver → **worklog.md** + **results/** (manifest.json · QN.sql · QN.csv · QN.meta.json) Validator → **validation/report.md** + 7 check files Presenter → **deliverable/** (README · ask-coverage · artifacts/; versioned -v2, -v3, never overwritten)

1 · SOLVER — resolve

WHAT

- 4 phases: triage (6 gates) → understand → plan → execute → deliver
- MCP for every business def — **never guesses**; no SQL before MCP

SELF-HEALING

- 10-retry budget (3 syntax · 2 timeout · 3 zero-row), capped
- Zero-row → Investigation Branch: hypothesis → 5-min probe → apply

STATE

- manifest.json (machine truth) + worklog.md (narrative) → crash & resume, no re-runs

GATE

Validator gates the presenter; I gate the final send. Today I'm the gate → next: golden-set regression.

2 · VALIDATOR — verify

WHAT

- Pure dispatcher — runs no SQL itself; launches **7 parallel sub-agents**, each an independent check (the eval, productized)

THE 7 CHECKS

- Completeness — WHO/WHAT/WHEN answered?
- Definitions vs MCP ★ **most critical**
- SQL quality — cost-safe, no anti-patterns
- Result verification — CSVs re-run vs live source
- Worklog quality — documentation useful?
- End-to-end coherence — one story?
- Process analysis — was MCP called before SQL?

VERDICT

- PASS / WARN / FAIL — gates the presenter

ABSTAIN

Grounded — cites only data the SQL returned. Budget 0 → partial + logged blockers, never a fabricated number.

3 · PRESENTER — package

WHAT

- Turns a validated result into a stakeholder deliverable — self-contained HTML

DECISIONS

- Validation gate**: refuses to build on a FAIL / stale result unless forced
- Re-extracts the **literal ask** from the ticket — not the solver's spin (drift check)
- Audience-adaptive: business vs engineer tone auto-detected

THEN

- Slop-scrub sub-agent strips AI filler; never overwrites originals

FEAR

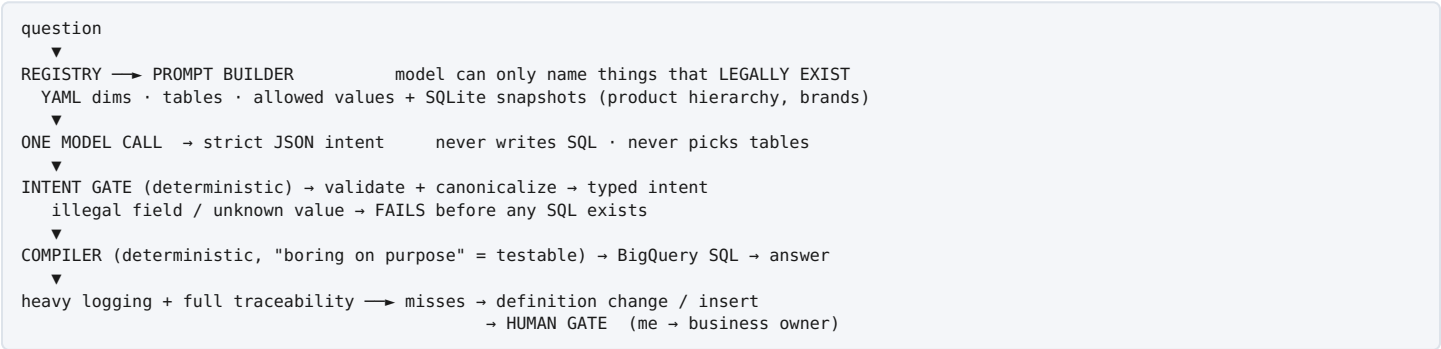
Confident-wrong — clean SQL on a stale flag. Hence freshness stamps (frozen 30d / late 24h / streaming 1h) + the validator's source re-run.

Leverage close: "The intake bottleneck was what kept me from building anything else. Automating my own role bought me the time to build the rest of the portfolio."
Autonomy posture: agent autonomous to a validated deliverable; my review sits OUTSIDE the loop — two failure budgets, each sized to blast radius. | If "draw it": 3 stages + substrate underneath, 60 sec, narrate every box. Pick 2–3 numbers max.

Natural-language analytics skill with the business logic moved into a **governed semantic layer** — **exactly one model call** in the path, boxed in on both sides by deterministic code, every new definition gated by a human.

Evolution: context-engineering (raw SQL) → semantic layer · **parallel reconciliation retired** → now **sequential + human gate**. Same question, same answer, every time.

THE ARCHITECTURE — DETERMINISTIC → PROBABILISTIC → DETERMINISTIC



THE PATH — ONE MODEL CALL, BOXED BOTH SIDES

STAGE	KIND	WHAT IT DOES
Registry → prompt builder	deterministic	Generates the model's contract straight from the governed registry — the LLM can only reference dimensions, tables, and values that legally exist.
One model call	probabilistic	The only probabilistic step. Returns strict JSON intent — never writes SQL, never picks tables. A semantic parser, boxed in.
Intent gate	deterministic	Validates + canonicalizes JSON into a typed intent . Illegal field or unknown value fails before any SQL exists .
Compiler	deterministic	Typed intent → BigQuery SQL. Boring on purpose — boring means testable like normal software.
UPC lane	deterministic	High-cardinality IDs never enter the prompt. Model flags " <i>this is a UPC,</i> " code normalizes the format + checks it exists — a fake UPC fails loudly instead of returning a bogus zero.

DESIGN DECISIONS — LEAD WITH THE ARCHITECTURAL REASON

- Why a semantic layer?** Context-engineering wrote raw SQL — same question could yield slightly different numbers, and business users notice. Definitions now live as governed semantic definitions; *determinism is the trust mechanism*.
- Exactly one model call.** The LLM maps question → definition and nothing else. Registry + gate **split the fuzziness**: "mael" → male is the model's job; is "Male" a legal value is the gate's. Two failure modes, two owners.
- Human gate on every definition.** New or changed definition → me first while I calibrate, then the **business owner of that metric**. The gate is the governance — the business owns the meaning of its own numbers.
- Parallel → sequential.** Ran both paths in parallel; a reconciliation agent graded the layer. But that's *2× compute in the request path for an offline problem*. Moved grading onto the logs — same loop, off the hot path.

WHY IT'S AN EVAL DECISION AS MUCH AS ARCHITECTURE

- The compiler is deterministic, so the eval surface **collapsed to one step** — question → definition mapping. That mapping **is** the intent question.
- The human gate is the labeling flywheel:** every correction is a labeled failure case that feeds the regression set. Eval data grows from real usage.
- Harness:** golden intent set (doubles as a cheap regression suite) · metrics = definition-selection accuracy, coverage/abstain rate, regression pass rate · LLM-as-judge as *triage only*, business owner as truth.

GATE

Human gate on every new/changed definition — me, then the business owner. Governance + the eval-dataset flywheel in one.

ABSTAIN

No matching definition → abstain or clarify, never guess. The prompt tells the model to omit any predicate it can't name from the registry.

FEAR

A fake UPC returning a bogus zero people learn to trust. High-card IDs fail loudly before SQL — separated from a valid zero.

The line: "The semantic layer was an eval decision as much as an architecture decision — if I can't write a cheap regression test for a system, that's an architecture smell. This is the testable version of the same capability."

If "draw it": the deterministic → probabilistic → deterministic spine, 60 sec. Money beat: **one model call, boxed both sides by the registry and the gate** — and every correction through the human gate is a labeled example.

CP Analytics — Autonomous Multi-Hypothesis Analyst

DEPTH RESERVE

An autonomous analysis orchestrator — turns a plain-English question into reproducible, audited SQL analyses, and **every finding spawns the next hypothesis**. A prototype for how far the context layer goes unsupervised.

50 ideas explored · prototype, **not yet in production** · runs on the **same cp-analytics MCP as the Jira agent** · next: generation moves onto the semantic layer

THE ARCHITECTURE — 4 SKILLS IN SEQUENCE, ALL ON THE SHARED CONTEXT LAYER

THE 4-SKILL PIPELINE — sequential; the analyst is a pipeline, like the Jira agent

- 1 cp-context-validate → regen schema from BigQuery; validate domain / tables → prevents drift
- 2 cp-analytics → orchestrate: each idea → isolated sub-agent → SQL + CSV + charts
REFLECT = Impact × Surprise × Gaps; score ≥ 4 → DEPTH DIVE → loop
- 3 cp-audit → 7-check validator → PASS / WARN / FAIL
- 4 cp-fix → auto-remediate findings → re-dispatch cp-audit
→ [MY REVIEW] → I'm still the final reader (prototype — not in production)

cp-analytics MCP schema · joins · business defs
· 52-node join graph (shared with the Jira agent)

SAME context layer as page 1

WHAT EACH SKILL DOES — ONE JOB EACH, RUN IN ORDER

1 · cp-context-validate

- Regenerates schema.json from BigQuery INFORMATION_SCHEMA on **every run**
- Checks domain / tables vs reality — columns, enum values, join paths, inventory → **prevents context drift**

2 · cp-analytics

- Bash-free orchestrator — each idea → **isolated sub-agent** runs BigQuery, writes SQL + CSV + charts
- REFLECT** (Impact × Surprise × Gaps) drives depth dives; empty backlog → generates 10 new hypotheses

3 · cp-audit

- 7 independent checks:** SQL schema · logic · interpretation · prose arithmetic · cross-analysis · evidence chain · reproducibility
- Verdict **PASS / WARN / FAIL** — gates remediation

4 · cp-fix

- Fixes in dependency order: SQL rewrite · recalc · artifact repair · narrative rewrite · chart regen
- Snapshots originals to .fix/pre_fix/, verifies each edit landed, **re-dispatches cp-audit**

DESIGN DECISIONS — LEAD WITH THE ARCHITECTURAL REASON

- Bash-free orchestrator.** Sub-agents do all execution; the orchestrator only coordinates — avoids CLI prompts on read-only SELECTs and isolates each idea's context so big results don't pollute another's reasoning.
- SQL traceability as a contract.** Every query stored as full, copy-paste-executable SQL — an idea *fails validation* without it. Every number traces to a runnable query.
- State in JSON, not memory.** session.json + backlog.json survive any crash; resume recovers mid-flight. Crash recovery is a primary feature.
- Single-responsibility, composable skills.** Each does one thing — so each is independently testable and swappable.

STATUS & WHERE IT'S HEADED

- Prototype, not production:** ~50 ideas explored end-to-end — proves the autonomy loop, not yet shipped.
- Today sub-agents **generate raw SQL** on the context layer.
- Next move:** route generation through the **semantic layer** (page 2) — the model stops writing SQL, generation goes deterministic. Same autonomy, trustworthy by construction.

GATE

cp-audit grades each analysis PASS / WARN / FAIL; cp-fix remediates. Today I'm still the final reader.

ABSTAIN

Grounded — cites only what the SQL returned. Blockers logged; never a fabricated number.

FEAR

Confident-wrong — clean SQL on a stale or mis-defined metric. The semantic-layer move (page 2) is exactly the fix.

Substrate through-line: "Same cp-analytics MCP as the Jira agent — the Jira agent proved the context layer in production, this shows how far it goes unsupervised, and the semantic layer is where its SQL generation is headed. One reusable layer, three thin skills on top."

If "draw it": the 4-skill pipeline (validate → analyze → audit → fix) feeding [MY REVIEW], on the shared MCP, 60 sec. Money beat: **every finding spawns the next hypothesis**, and audit → fix → re-audit makes it self-correcting — not just self-extending.